GLU Variants Improve Transformer

Noam Shazeer (2020), arXiv:2002.05202

Paper summary for the CS-781 capstone

The paper at a glance

Citation

N. Shazeer. *GLU Variants Improve Transformer*. arXiv preprint, February 2020. Google.

One-sentence summary

Replace the Transformer's ReLU feed-forward layer with a **gated** version, such as SwiGLU or GEGLU, at equal parameters and compute. Perplexity and downstream scores improve.

Why it matters for our capstone

Our A4 uses exactly the paper's parameter-matched SwiGLU: $d_{\text{model}} = 768$, $d_{ff} = 2048$, three matrices, no biases.

Where it appears in our slides

Main deck: "A4: efficient decoder". Glossary: "Activation functions" (SwiGLU as a valve).

Roadmap

1. Background
2. The basic idea: GLU variants in the FFN
3. Experiments
4. Assessment and outlook

The feed-forward network (FFN) in a Transformer

Each Transformer block has attention (tokens exchange information) and an **FFN**, applied to each token on its own.

Original (Vaswani et al., 2017), and the bias-free T5 version used here:

$$\mathrm{FFN}_{\mathrm{ReLU}}(x) = \max(xW_1, 0)\, W_2$$

Other activations had been proposed:

$$\mathrm{GELU}(x) = x\,\Phi(x), \qquad \mathrm{Swish}_\beta(x) = x\,\sigma(\beta x)$$

About two thirds of a Transformer block's parameters sit in the FFN, so its design matters.

Intuition

Attention gathers facts from other tokens, e.g. "he" refers to "respondent". The FFN is where each token *processes* what it gathered: a learned bank of feature detectors.

Symbols

$x \in \mathbb{R}^d$: one token's vector. $W_1 \in \mathbb{R}^{d \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d}$. Typically $d_{ff} = 4d$.

Gated Linear Units (Dauphin et al., 2016)

A **GLU** multiplies two linear projections element-wise; one passes through a sigmoid:

$$\text{GLU}(x) = \sigma(xW + b) \otimes (xV + c)$$

- xV: the *content*.
- $\sigma(xW)$: a *gate* between 0 and 1 deciding how much content passes.

Dropping the sigmoid gives the **bilinear** layer:

$$\text{Bilinear}(x) = (xW + b) \otimes (xV + c)$$

Analogy: a valve on a pipe

Water (content) flows through a pipe; a valve (gate) opens or closes. Both the water and the valve setting depend on the input. A plain ReLU is a one-way valve that can only be fully shut (negative) or open in proportion (positive).

The proposed variants

Replace the first linear layer and activation of the FFN with a GLU variant (no biases):

$$\begin{aligned}
\mathrm{FFN}_{\mathrm{GLU}}(x) &= (\sigma(xW) \otimes xV)\, W_2 \\
\mathrm{FFN}_{\mathrm{Bilinear}}(x) &= (xW \otimes xV)\, W_2 \\
\mathrm{FFN}_{\mathrm{ReGLU}}(x) &= (\max(0, xW) \otimes xV)\, W_2 \\
\mathrm{FFN}_{\mathrm{GEGLU}}(x) &= (\mathrm{GELU}(xW) \otimes xV)\, W_2 \\
\mathrm{FFN}_{\mathrm{SwiGLU}}(x) &= (\mathrm{Swish}_1(xW) \otimes xV)\, W_2
\end{aligned}$$

$\mathrm{Swish}_1(z) = z\,\sigma(z)$, also called SiLU.

Fair comparison: equal parameters

GLU variants have **three** matrices (W, V, W_2) instead of two. To keep parameters and compute equal, shrink d_{ff} by $\frac{2}{3}$:

$$2 \cdot d \cdot 3072 \;=\; 3 \cdot d \cdot 2048$$

For $d = 768$: 4.72M parameters per FFN in both cases.

Our A4

`SwiGLU(d_model=768, d_ff=2048)`: identical to the paper's matched setting.

Easy worked example (2 hidden units)

Suppose for one token: gate pre-activation $xW = [2, -1]$ and content $xV = [3, 4]$.

Layer	Hidden vector (before W_2)
ReLU (only xW)	$[2, 0]$
GLU	$\sigma(xW) \otimes xV = [2.64, 1.08]$
Bilinear	$xW \otimes xV = [6, -4]$
ReGLU	$\max(0, xW) \otimes xV = [6, 0]$
SwiGLU	$\text{Swish}(xW) \otimes xV = [5.28, -1.08]$

$\sigma(2) = 0.88$, $\sigma(-1) = 0.27$; $\text{Swish}(2) = 2\sigma(2) = 1.76$, $\text{Swish}(-1) = -0.27$.

What to notice

- In ReLU, unit 2 is simply off.
- In gated layers, the output is a *product* of two input-dependent signals. The network can express "this feature matters only when that one is present".
- SwiGLU lets a small negative value through (smooth), unlike ReGLU's hard zero.

Setup

- Same code, model and task as the T5 base model: encoder–decoder, 12+12 layers, $d_{\text{model}} = 768$, 12 heads, $d_k = d_v = 64$.
- FFN: $d_{ff} = 3072$ for ReLU, GELU and Swish; $d_{ff} = 2048$ for the GLU variants.
- Pre-training: span-filling on C4, 524,288 steps, 128 examples $\times$ 512 input tokens per batch; Adafactor, inverse-square-root schedule, linear decay over the last 10%.
- **No dropout during pre-training**, a departure from T5 that gave better results.
- Four short runs (65,536 steps) per variant measure run-to-run noise.

Measures

- Held-out **log-perplexity** on the span-filling task (C4). Lower is better.
- Fine-tuning on a mixture of GLUE, SuperGLUE and SQuAD; development-set scores.

Cost

Each training step took about 0.15 s on a 32-core TPUv2 cluster. All variants cost the same by design.

Pre-training perplexity

Held-out log-perplexity (lower is better); standard deviation over 4 runs in brackets.

FFN variant	65,536 steps	524,288 steps
ReLU (baseline)	1.997 (0.005)	1.677
GELU	1.983 (0.005)	1.679
Swish	1.994 (0.003)	1.683
GLU	1.982 (0.006)	1.663
Bilinear	1.960 (0.005)	1.648
GEGLU	**1.942** (0.004)	**1.633**
SwiGLU	1.944 (0.010)	1.636
ReGLU	1.953 (0.003)	1.645

Reading it

- Changing only the activation (GELU, Swish) barely helps.
- **Every gated variant beats every ungated one.**
- GEGLU and SwiGLU are best: about 0.04 lower than ReLU after full training, far above the run-to-run noise of about 0.005.
- Even the bilinear layer (no non-linearity in the gate) beats ReLU.

Downstream fine-tuning (development sets)

FFN variant	GLUE	SGLUE	SQuAD F1
ReLU	83.80	72.76	90.87
GELU	83.86	72.98	90.79
Swish	83.60	72.40	90.76
GLU	84.20	73.95	90.69
GEGLU	84.12	73.96	91.12
Bilinear	83.79	73.81	91.06
SwiGLU	84.36	**74.56**	91.03
ReGLU	**84.67**	73.66	**91.18**
T5 paper baseline	83.28	71.36	88.81

T5's reported standard deviations: GLUE 0.235, SuperGLUE 0.416.

Reading it

- GLU variants are best on most tasks, e.g. SuperGLUE 74.56 (SwiGLU) vs 72.76 (ReLU).
- The results are noisy: each model was fine-tuned only once.
- The paper's ReLU model beats the original T5 numbers (83.80 vs 83.28). The author attributes this to removing dropout during pre-training.

Relevance to our capstone

- A4 uses SwiGLU; A1 and A3 use GELU; A2 uses ReLU. This makes the FFN one of the *differences* between our architectures.
- The paper predicts a small but real perplexity gain for A4 from the FFN alone.
- Our default `dropout: 0.0` matches the paper's finding that no dropout helps pre-training when data is plentiful.

A controlled ablation you could run

Train A1 twice, identical except the FFN:

- GELU, $d_{ff} = 3072$
- SwiGLU, $d_{ff} = 2048$

Same parameters, same data. Compare validation loss and dev BERTScore. This separates the FFN effect from A4's other changes (GQA, RoPE, sliding window).

Strengths and limitations

Strengths

- A tiny code change with consistent gains.
- A fair comparison: parameters and compute matched.
- Measures run-to-run noise for perplexity.
- No extra cost at training or inference.
- Adopted widely in later large models.

Limitations

- **No explanation** of why gating works. The author jokingly credits "divine benevolence".
- One model size and one architecture (T5 base).
- Fine-tuning results come from one run each, and are noisy.
- Pre-training changes (no dropout) are mixed into the comparison with T5's numbers.
- A very short paper, with no ablation of d_{ff} or scaling behaviour.

Future research directions

What followed

- SwiGLU became standard in large language models, e.g. PaLM (Chowdhery et al., 2022) and LLaMA (Touvron et al., 2023).
- GEGLU is used in the T5 v1.1 checkpoints and in the Gemma models.
- Sparse FFNs: mixture-of-experts layers (e.g. Switch Transformer, Fedus et al., 2022) replace one FFN with many expert FFNs, and only a few run per token.

Open questions

- **Why** does multiplicative gating help: better optimisation, or more expressive functions?
- Does the gain grow, shrink or vanish at larger scale?
- What is the best d_{ff} ratio for gated FFNs?
- Do gated FFNs store factual knowledge differently?
- For legal NLP: do gated FFNs help with rare legal terms, or with long formulaic sentences?

Summary

- The Transformer FFN is $\max(xW_1, 0)W_2$. Changing only the activation (GELU, Swish) gives little.
- **Gated** FFNs multiply a content projection by an activated gate projection: GLU, Bilinear, ReGLU, GEGLU, SwiGLU.
- At equal parameters (d_{ff} reduced by $\frac{2}{3}$), all gated variants lower pre-training perplexity; GEGLU and SwiGLU are best (1.633 and 1.636 vs 1.677).
- Downstream scores improve on most tasks, e.g. SuperGLUE 74.56 vs 72.76.
- For us: the reason A4 uses SwiGLU with $d_{ff} = 2048$.

Reference

Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv:2002.05202.

Other work cited: Dauphin et al. (2016) Language Modeling with Gated Convolutional Networks; Hendrycks and Gimpel (2016) GELU; Ramachandran et al. (2017) Swish; Raffel et al. (2020) T5; Chowdhery et al. (2022) PaLM; Fedus et al. (2022) Switch Transformers; Touvron et al. (2023) LLaMA.